

Audit Report

Own Protocol v2

Date: September 2 – September 9, 2026

Introduction

Own is a permissionless protocol for bringing tokenized real-world assets (RWAs) onchain. Users mint ERC-20 tokens called eTokens (e.g. eTSLA, eGOLD) using stablecoins; each eToken tracks the price of its underlying asset through onchain oracles and is backed by on-chain collateral held in Own Vaults. Where traditional RWA tokenization concentrates trust in a single custodian holding the physical asset, Own backs exposure with transparent, code-governed collateral vaults: any hedge fund, trading firm, or RWA custodian can permissionlessly operate a vault and back issued eTokens, with competition between them driving spreads.

Minting and redemption flow through a quote-driven RFQ marketplace: a buyer requests a firm off-chain quote from a market maker and either settles it atomically as a market order (`executeOrder`) or rests a limit order (`placeOrder`) for a maker to fill. Quotes are EIP-712 signed and authorized against a global signer registry, and settle inside a price band anchored to the protocol's oracle marks. Every holder also carries a code-enforced exit: a redeemer who cannot obtain a quote can force-execute against vault collateral at the oracle price after a claim window. Liquidity providers deposit collateral into ERC-4626 `OwnVaults` and share the fees and yield the vault earns; a `BorrowManager` additionally lets users borrow stablecoins against their eToken collateral.

This fourth review covers the eUSD stablecoin module introduced on this branch. `EUUSDManager` is a CDP facility where users lock eTokens as collateral and mint eUSD, an overcollateralized dollar stablecoin: positions sit in a sorted list ordered by collateral ratio, with minting ratios, keeper liquidation at a fixed bonus, and Liquity-style redemption against the riskiest positions anchoring the peg. `EUUSD` is the token, with rate-limited ERC-7802 bridge entry points; `StakedEUUSD` is an ERC-4626 vault streaming protocol yield to stakers with linear vesting; `OwnIncentives` distributes OWN rewards to sEUUSD holders via a balance-checkpoint hook. `OwnMarket` also gained a UUPS upgrade path, with force-execution extracted into `ForceExecuteLib`.

The review combined manual analysis of every contract in scope with independent multi-agent adversarial audit passes – module-scoped, then branch-wide, then protocol-wide – with every finding adversarially validated against source. The module's core accounting was attacked directly and held: eUSD supply conservation, sorted-list integrity, staking vesting continuity, and the incentives index algebra all survived. Every High and Medium fix carries a regression test verified to fail without it. Full suite: 1,430 tests passing.

Scope

Repository: `own-protocol/own-v2`

Audited Commit: `c1cb8e7747230cd1413081129a4caebfb948f3fc`

Files:

- `src/core/EUSDManager.sol`
- `src/core/OwnIncentives.sol`
- `src/core/OwnMarket.sol`
- `src/libraries/ForceExecuteLib.sol`
- `src/tokens/EUSD.sol`
- `src/tokens/StakedEUSD.sol`

The primary scope is the six contracts new or materially changed on this branch. In addition, two full protocol-wide passes re-audited all 25 `src/` contracts and re-checked every finding from the three prior reports against current source: no previously fixed finding has regressed, with one exception – the expiry half of Report 3's M-11 was found never implemented and is fixed in this pass as L-15.

Findings

Each issue has an assigned severity:

Critical/High issues are directly exploitable security vulnerabilities that can lead to loss of funds and need to be fixed.

Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.

Low/Info issues are non-exploitable or informational findings that do not pose a direct security risk to the system's integrity.

Issues marked **RESOLVED** are fixed in the codebase and carry a regression test that fails without the fix. Issues marked **ACKNOWLEDGED** are accepted by design, with the

rationale and any operational mitigations documented. Two findings raised as Medium (L-01, L-02) were reassessed to Low on validation and are counted as Low below.

Issues Found

Critical	High	Medium	Low	Info
0	2	6	21	20

Issues Not Fixed and Not Acknowledged

Critical	High	Medium	Low	Info
0	0	0	0	0

Summary of Findings

ID	Severity	Title	Status
H-01	High	Partial redemption of an underwater position stranded unbacked debt at the list head	Fixed
H-02	High	A routine stock split silently mis-valued every eUSD position by the split ratio	Fixed
M-01	Medium	Redemption could not skip an underwater head, stalling the peg anchor	Fixed
M-02	Medium	OwnIncentives paid retroactive OWN on balances from unhooked windows	Fixed
M-03	Medium	Full-debt-only liquidation could be starved of eUSD liquidity	Fixed

M-04	Medium	Halted collateral was valued at the live feed, enabling unbacked minting	Fixed
M-05	Medium	Force-execute on a PSM-backed asset: vault LPs pay while the maker collects surplus	Acknowledged
M-06	Medium	Partial liquidation below 1 + bonus seized the full bonus, manufacturing unbacked eUSD	Fixed
L-01	Low	Window-open premium is caller-timed but self-corrects on the next <code>accrue</code>	Acknowledged
L-02	Low	JIT yield capture when the best-effort claim is blocked at the pool LTV	Acknowledged
L-03	Low	<code>mintPriceMaxAge</code> was a silent no-op inside the oracle's <code>clFreshWindow</code>	Fixed
L-04	Low	Sorted-list ordering drifts under lazy stability-fee accrual	Acknowledged
L-05	Low	No <code>minDebt</code> floor on the redemption path	Acknowledged
L-06	Low	Oracle <code>maxAnchorAge</code> width vs liquidation bonus (stale-anchor exits)	Acknowledged
L-07	Low	<code>setRiskParams</code> threshold raise assumes ADMIN sits behind the timelock	Acknowledged
L-08	Low	ADMIN/OPERATOR role namespace is protocol-global, not per-contract	Acknowledged
L-09	Low	<code>MINTER_ROLE</code> exclusivity not structurally enforced on EUSD	Fixed
L-10	Low	eToken collateral dividends were stranded in the manager with no claim path	Fixed
L-11	Low	<code>setPartner</code> + permissionless sweep can redirect a holder's accrued OWN	Acknowledged

L-12	Low	Disabled collateral blocked defensive top-ups while liquidation stayed live	Fixed
L-13	Low	Rewards streamed into an empty sEUSD vault were silently donated	Fixed
L-14	Low	Bridge burn below the unvested slice: vault DoS, then a share-capture leg	Fixed
L-15	Low	Force-execution never checked <code>order.expiry</code> (Report 3's M-11's unimplemented half)	Fixed
L-16	Low	<code>withdrawCollateral</code> with debt escaped the <code>mintPaused</code> lever	Fixed
L-17	Low	A code-less incentives controller bricked every sEUSD transfer	Fixed
L-18	Low	<code>haltAsset</code> price is operator-set, unbounded, and permanent	Acknowledged
L-19	Low	Pending-deposit escrow counted as vault collateral by pool health gates	Acknowledged
L-20	Low	Rate setters reprice the elapsed accrual window (no accrue-first)	Acknowledged
L-21	Low	PSM ratio under signer compromise: damage $\approx 2 \times$ <code>bandBps</code> , jump guard walkable	Acknowledged
I-01	Info	<code>_freshPrice</code> tolerates future-dated timestamps	Acknowledged
I-02	Info	Fee rounds to zero but <code>feeIndexSnapshot</code> still advances	Acknowledged
I-03	Info	Zero-fee redemption (no Liquity-style base rate)	Acknowledged
I-04	Info	<code>debt * (BPS+bonus)</code> computed outside <code>mulDiv</code> 's 512-bit space	Acknowledged

I-05	Info	<code>EUSD.crosschainBurn</code> burns from an arbitrary <code>from</code> (trusted-bridge model)	Acknowledged
I-06	Info	Bridge burn+mint pairing evades the <code>netBridgedIn</code> global cap	Acknowledged
I-07	Info	OWN sent directly to <code>OwnIncentives</code> (not via <code>fund</code>) is unrecoverable	Acknowledged
I-08	Info	<code>recoverReserve</code> can pull reserve backing accrued-but-unclaimed OWN	Acknowledged
I-09	Info	Zero-amount <code>crosschainMint/Burn</code> let any EOA emit spoofed bridge events	Fixed
I-10	Info	<code>setBridgeLimits</code> reset remaining capacity to max – instant window refill	Fixed
I-11	Info	Stability-fee accrual bypasses <code>debtCeiling</code> (Maker-style, intended)	Acknowledged
I-12	Info	<code>_verifyInhouseProof</code> lacks the per-asset staleness bound of <code>updatePrice</code>	Acknowledged
I-13	Info	<code>ReserveVault.skimExcess</code> pays surplus to <code>msg.sender</code> (hot key)	Acknowledged
I-14	Info	<code>LendingRouter.deposit</code> bypasses the pool supplier allowlist via a shim vault	Acknowledged
I-15	Info	<code>EToken.updateName</code> breaks the cached ERC-2612 permit domain	Acknowledged
I-16	Info	<code>_lastPremiumBps</code> zero-sentinel collides with a real 0 observation	Acknowledged
I-17	Info	<code>setChainlinkConfig</code> keeps the old in-house price cache	Acknowledged

I-18	Info	<code>OwnIncentives.setDistribution</code> emission is unbounded	Acknowledged
I-19	Info	Enabling a zero side of a live bridge starts it empty (fail-safe)	Acknowledged
I-20	Info	<code>setIncentivesController(current)</code> retires the live controller in place	Acknowledged

Detailed write-ups follow for every fixed finding and for the acknowledged findings whose trust boundaries are load-bearing. The remaining acknowledged Low/Info items are catalogued with their rationale in the in-repo remediation document.

Issue H-01: `_redeemFrom` stranded unbacked zero-collateral debt at the list head RESOLVED

Severity: **High**

Vulnerability Detail

`_redeemFrom` capped `seized` at `p.collateral` but subtracted the full `repaid` from `p.debt`, so partially redeeming an underwater position left `debt > 0`, `collateral == 0`. The surviving node re-entered the sorted list with key `collateral/debt = 0` – the minimum – pinning it permanently at `listHead`. Every subsequent `redeem` had to consume the head first: `seized` capped at zero, so redeemers burned eUSD for no collateral, and any redeemer with a non-zero `minCollateralOut` reverted `SlippageExceeded` and could not redeem at all. `liquidate` could not clear it (burning full debt for zero collateral is a pure loss no keeper takes) and the owner had no incentive to `closePosition`. The state is reachable by any redemption of amount in `[collateral value, debt)` against an underwater position, and an attacker could manufacture it deliberately at fair-value cost, with the toll landing on everyone after them. The root cause was an asymmetry: `liquidate` handles the same collateral cap by clearing the *full* debt, `_redeemFrom` did not.

Impact

A permanent toll – or a hard revert under slippage protection – on the peg-defense redemption path, plus unbacked eUSD accumulating in `totalDebt`.

Resolution

`_redeemFrom` now caps `repaid` to the seized collateral's value at the redemption price whenever the collateral cap fires, so a redeemer is never charged for collateral they do not receive. The unbacked residual stays on the owner's books, off-list, still counted in `totalDebt` (the supply invariant is untouched), and clearable by repay, close, liquidation, or top-up. List membership is now link-derived (`_isListed`) instead of inferred from `debt > 0`; `_reindex` inserts only when `debt > 0 && collateral > 0`; and the `_insertNode` hint check ignores stale hints pointing at off-list residuals, which previously could link a new node behind a detached predecessor with a caller-

supplied hint. The redemption walk continues past a drained head in the same call. Five regression tests cover the zombie path, fair-value walk continuation, and residual re-listing, and the list invariant now asserts no listed node has zero collateral.

Issue H-02: A routine stock split silently mis-valued every eUSD position by the split ratio

RESOLVED

Severity: **High**

Vulnerability Detail

`EUSDManager` custodied collateral as a fixed token-address balance but valued it by ticker: every ratio and seizure computation multiplied the raw stored unit count by the per-active-unit ticker price, applying no legacy-ratio factor, and the module had no split/migration hook. When the admin performs a supported corporate action – `AssetRegistry.migrateToken(ticker, newToken, ratio)` – the deposited token becomes legacy, the external feed reports the new per-active-unit price, and every existing position keeps operating silently mispriced by `ratio`. Worked case (MCR 150%, threshold 130%, bonus 5%): a position of 2 eSPY against 800 eUSD at \$600/share is worth \$1,200 (CR 150%). A 2:1 forward split moves the feed to \$300; the position reads 75% and any keeper liquidates it, receiving \$1,200 of true value for \$800 – a 50% profit, with the borrower losing the entire MCR buffer. A reverse split inverts it: positions over-valued by `ratio`, allowing withdrawal or fresh minting against phantom backing. `addCollateral` also accepted already-legacy tokens, so the mispricing was reachable without any migration, and `BorrowManager._effectivePrice` already implemented the correct scaling – the manager was the sole eToken pricer without it.

Impact

A legitimate, protocol-supported corporate action made genuinely healthy positions liquidatable at a large keeper profit (forward split) or admitted uncollateralized eUSD supply (reverse split), with the victim's `withdrawCollateral` rescue blocked by the same wrong price.

Resolution

A new private `_effectivePrice(collateral, price)` scales the per-active-unit ticker price by `legacyRatioToActive(collateral)` (ratio 0 → identity), mirroring the existing `BorrowManager` house pattern. It is applied inside the two price helpers

`_freshPrice / _anchorPrice`, so every valuation site – withdraw, mint, liquidate, redeem, ratio views – is covered by construction, while seizure and transfer amounts stay denominated in the token actually held. `addCollateral` now rejects already-legacy tokens (`LegacyCollateral`). The migrate-before-feed ordering, mint/deposit freeze, and legacy-disable steps are recorded as the eUSD collateral policy & split runbook. Six unit tests plus an integration suite against the real `AssetRegistry.migrateToken + VaultManager.applySplit` cover forward, reverse, and two-hop migrations.

Issue M-01: Redemption could not skip an underwater head, so the peg anchor stalled exactly when liquidation was unprofitable

RESOLVED

Severity: **Medium**

Vulnerability Detail

`redeem` consumes strictly from `listHead` with no skip mechanism. Any head with $CR < 100\%$ short-changed redeemers, and a redeemer protecting themselves with an accurate `minCollateralOut` always reverted and could not reach healthy positions behind the head. This bites precisely in the crash regime where liquidating that head ($CR < 100\% + \text{bonus}$) is also loss-making, so no rational actor cleared the blockage.

Impact

The peg-defense redemption path was inert exactly while eUSD traded below par.

Resolution

Closed by the H-01 fix without a separate change. An underwater head is now redeemed value for value (the burn is capped at its collateral's worth), it leaves the list in the same call, and the walk continues to the next position at full value – a redeemer receives exactly one dollar of collateral per eUSD across the walk and an accurate `minCollateralOut` holds. Pinned by a regression that redeems through an underwater head into a healthy position with an exact non-zero `minCollateralOut`. The unbacked residual left on the drained owner's books is tracked under H-01 as a backstop/accounting decision, not a liveness issue.

Issue M-02: OwnIncentives paid retroactive OWN on sEUSD balances acquired while the hook was detached

RESOLVED

Severity: **Medium**

Vulnerability Detail

`claim`, `sweepPartner` and `earned` read the holder's live `balanceOf` and applied the full index delta since the holder's snapshot, on the stated assumption that every supply change was checkpointed by the sEUSD hook. That assumption was enforced nowhere: `StakedEUSD.incentivesController` defaults to zero, was admin-clearable and swappable at any time, and the hook call is `try/catch`-wrapped. During any window in which the controller was not attached – deploy mis-sequencing or a mid-campaign swap – sEUSD balances moved with no checkpoint, so a balance acquired in the window claimed the entire historical index at its current size, and the same sEUSD could be shuttled across fresh addresses and claimed from each until the reward reserve drained. PoC: with the campaign live and the controller detached for a day, an attacker minted 1,000,000 sEUSD, claimed $\approx 43,200$ OWN for zero seconds of holding, shuttled to a fresh address, and claimed the same amount again – more than the day's emission.

Impact

Reserve drain and dilution of honest holders whenever a wiring or migration window existed; later claimers hit `RewardShortfall`.

Resolution

Freezing while detached alone was judged insufficient (it only defers the replay to re-attach), so every seam is closed: `OwnIncentives` freezes accrual whenever it is not the wired controller and pays only already-accrued OWN; `setDistribution` with non-zero emission reverts `NotAttached` unless wired; and `StakedEUSD.setIncentivesController` permanently retires the outgoing controller so a detached index can never be replayed, and rejects code-less targets (closing L-17 with the same guards). The planned OWN-migration replacement lever and the

emergency clear-to-zero both remain available, and the wiring/decommission order is recorded as a runbook. Five tests cover attach ordering, the shuttle-claim PoC (now pays nothing), migration, and the setter guards.

Issue M-03: Full-debt-only liquidation could be starved of eUSD liquidity RESOLVED

Severity: **Medium**

Vulnerability Detail

`liquidate` burned the position's entire debt from the caller in one call, with no partial option. A large borrower could therefore make themselves structurally unliquidatable: mint near the `debtCeiling` and hoard or bridge out the minted eUSD. When the position turned unsafe, no other actor could assemble the full debt – circulating supply outside the whale was smaller than the debt, minting liquidation liquidity was blocked by the ceiling the whale filled (`DebtCeilingExceeded`), redemption could only shave with whatever circulated, and raising the ceiling is a delayed ADMIN action spanning the crash. Worked case: ceiling 1M, whale mints 800k against \$1.2M collateral and hoards; a 40% drop leaves \$720k backing 800k debt and no path to burn 800k in one call.

Impact

The largest positions were unliquidatable in exactly the state liquidation exists for.

Resolution

Partial liquidation: `liquidate(collateral, owner, amount, hint)` repays `min(amount, debt)` (max = full close) with the same bonus math pro-rata, capped at the collateral. A partial that would leave $0 < \text{remaining} < \text{minDebt}$ reverts `BelowMinimumDebt`, the remainder is re-sorted with the caller's hint, and surplus collateral refunds only on a full close. Tests include a 10,000-debt whale cleared in chunks by a keeper who never holds more than 1,000 eUSD, and the invariant handler now liquidates partially whenever a keeper cannot fund the full debt. (The partial-seizure bonus math this introduced was subsequently hardened under M-06.)

Issue M-04: Halted collateral was valued at the live feed, enabling unbacked minting above the halt price RESOLVED

Severity: **Medium**

Vulnerability Detail

`EUSDManager` contained no reference to `VaultManager` halt or pause state. When an asset is halted, its eToken's only realizable value is the frozen `assetHaltPrice` (`OwnMarket.redeemHalted`), but the Chainlink feed keeps printing the live stock price. If the live price ran above $MCR \times \text{halt price}$, an attacker could buy halted eTokens near halt value, deposit them, mint at the live valuation, and abandon the position – the eUSD unbacked, and no rational liquidator clearing it since the seized eTokens redeem only at the halt price. Worked case: halt \$100, live rallies to \$160, MCR 150% → 1,000 eTokens bought for ~\$100k mint 106,666 eUSD, +\$6,666 per 1,000 tokens, scaling with the rally. A second wing surfaced in re-review: once a halted asset's feeds died past `maxAnchorAge`, every exit for that collateral bricked while debt remained outstanding.

Impact

Unbacked eUSD minted against halted collateral; more softly, every existing position in a halted asset over-valued whenever the live print exceeded the halt price.

Resolution

Both wings are closed in the two price helpers. `_freshPrice` (mint, withdraw-with-debt) reverts `CollateralHalted` when `VaultManager.isAssetHalted` is set. `_anchorPrice` (liquidate, redeem, ratio views) returns the legacy-ratio-scaled `assetHaltPrice` for a halted asset without consulting the oracle – the halt price is the eToken's only redeemable value, matching `BorrowManager.settleHaltedPosition`, and skipping the feed keeps exits alive after it dies. Deposits stay allowed for defensive top-ups, and repay / close / liquidate / redeem wind positions down with holders routed to `redeemHalted`. A same-day hardening also gates mint and withdraw-with-debt on `isTradingPaused`, with exits never pause-gated. Six unit tests plus an

integration test against the real `VaultManager.haltAsset`, including the worked attack case and a dead-feed exit case settling exactly at the halt price.

Issue M-05: Force-execute on a PSM-backed asset lets the maker collect freed reserve surplus while vault LPs pay

ACKNOWLEDGED

Severity: **Medium**

Vulnerability Detail

`closeExposure` on a force-executed redeem frees a reserve slice into the maker-withdrawable surplus (`ReserveVault.withdraw`) while the payout itself came from an allowlisted generic vault – the vault's LPs pay, the maker collects. A round-3 re-review widened the boundary: an allowlisted maker can also sign a Mint quote with `quote.user` set to its own linked address and call `executeOrder` – the settlement moves stablecoin maker → maker (zero net cash), `openExposure` passes under RWA netting, and the maker receives freshly minted eTokens it can `psmRedeem` against the wrapper reserve. Capital-free, and requiring neither an armed `claimThreshold` nor an allowlisted force-source vault.

Impact

The PSM reserve is only as safe as every allowlisted maker's quote-signing key; with force-execution armed on a PSM-backed asset, generic-vault LPs are the counterparty to maker non-performance.

Resolution

Accepted – makers are trusted, admin-allowlisted counterparties with exclusive per-asset quoting (`setMakerAllowed`); non-performance is a trust-model breach handled by de-allowlisting, the same trust that already underwrites the RFQ channel and reserve custody. Live state at review: force-execution is disabled globally (`claimThreshold == 0`). The acceptance is recorded with its true boundary, and an ops note: if force-execution is ever armed on an asset that also has a PSM reserve, keep the force-source vault pairing deliberate.

Issue M-06: Partial liquidation below 1 + bonus seized the full bonus, manufacturing unbacked eUSD RESOLVED

Severity: **Medium**

Vulnerability Detail

`_seizure` computed `seized = repaid × (1 + bonus) / price` and capped it at the position's whole collateral; on a partial, the remainder stayed on the books with no refund. The liquidator chooses the amount, so on any solvent position with ratio in $[100\%, 100\% + \text{bonus})$ a partial took the full bonus out of collateral the remaining debt needed. PoC (MCR 150% / threshold 130% / bonus 5%): at ratio 101%, a full close pays the keeper +\$10; a 900-eUSD partial pays +\$45 and collapses the remainder to 65%, after which a follow-on redemption leaves 35 eUSD of permanently unbacked, off-list supply. On an underwater position the same sizing *enlarged* bad debt by the bonus (a 95% position: full close −\$50, partial +\$45 with \$95 stranded). The partial strictly dominated the full close across the whole strip, there was no `owner != msg.sender` check (so debtors could self-liquidate the same way), and overnight gaps into the band are routine for single-stock eTokens with the anchor frozen off-hours.

Impact

Solvent-band liquidations manufactured unbacked eUSD and over-penalized borrowers; underwater partials increased bad debt versus the full close the shortfall-absorption rule assumed.

Resolution

`_seizure` now takes the position's debt and, on a partial, caps the seizure at the pro-rata share `coll × repaid / debt`. The cap is inert while the ratio is at or above $1 + \text{bonus}$ – the bonus seizure is already below pro-rata there, so M-03's chunked-liquidation liveness is unchanged – and below it the liquidator's bonus degrades to the position's cushion, so the remainder's ratio can never fall below the pre-liquidation ratio and a partial can no longer strand debt a full close would have covered. The interface's shortfall-absorption rule now holds for partials too. Both regression tests (in-

band pro-rata cap, underwater no-extra-bad-debt) fail without the cap; the existing 120% partial test pins the inert case.

Issue L-01: The premium observed at window open is caller-timed, but self-corrects on the next `accrue`

ACKNOWLEDGED

Severity: **Low** (raised as Medium, reassessed on validation)

Vulnerability Detail

Report 3's H-02's remedy bills each accrual window at `_lastPremiumBps`, the utilization premium observed when the window opened, so a denominator move can no longer reprice elapsed time – but the observation itself was caller-timed. Denominator side: deposit → mark pull → `accrue()` → same-block withdrawal leaves the deflated premium recorded while the real mark is restored, and the next window bills at it (PoC: \$53,276 of LP premium erased on a book left untouched for 30 days). Numerator side: every debt-mutating path observes before mutating, so a sole borrower repaying in full and re-borrowing next transaction records the zero-utilization base premium for its first window. Both mechanisms share one root: the observation is taken at a moment the caller controls and not re-taken after the caller's own state change.

Impact

Mispricing of a single inter-touch accrual window in either direction, shared pro-rata across the whole book.

Resolution

Reassessed to Low and acknowledged. `_accrue` re-observes the true utilization on every call with elapsed time – any keeper crank (permissionless, routinely cranked, the same cadence the Report 3's H-02 fix already relies on), or any borrow, repay, liquidate, or LP flow – so the mispriced premium survives only until the next touch of the book. Verified on the integration fixture: a single `accrue()` fifteen minutes after the attack restores the honest debt to the wei. The leak does not compound, must be re-armed with gas every window, and is not captured by the attacker alone. Optional hardening (bill `max(open, close)` per window) is recorded should the keeper cadence ever be relaxed.

Issue L-02: JIT yield capture when the best-effort premium claim is blocked at the pool LTV ACKNOWLEDGED

Severity: **Low** (raised as Medium, reassessed on validation)

Vulnerability Detail

The Report 3's M-01 remedy runs `_syncLending()` (accrue → best-effort claim → distribute) before pricing LP shares, so accrued premium is realized before a newcomer is priced. But the claim draws the premium as new vault debt and is refused once pool-side debt sits at the pool LTV; because the hook runs before the depositor's aTokens arrive, in exactly that state it is a no-op: the newcomer is priced pre-yield, their own deposit supplies the headroom, and a follow-up `syncYield()` distributes the whole accumulated gap pro-rata. PoC at Robinhood params: after 60 days at cap, a same-block deposit-sync-exit captured \$41,420 of yield earned entirely before the attacker arrived. The async `requestDeposit/acceptDeposit` path is immune (its escrow already counts as headroom).

Impact

Capture of pre-entry LP yield in the blocked-claim state, diluting incumbent LPs.

Resolution

Reassessed to Low and accepted. The payoff is the unclaimed premium backlog at deposit time, and that backlog does not realistically accumulate: every deposit, withdrawal, borrow and repay runs `_syncLending`, so any touch that lands with borrowing headroom realizes the accrued premium to existing LPs continuously, and an incumbent who notices a stuck gap can self-defend with a dust deposit plus `syncYield`. The large PoC figure needs the vault pinned at its debt cap with two months of unclaimed premium and open permissionless deposits – a combination the live config does not present (approval mode, 8-hour withdrawal queue, no atomic in-and-out). Escrow-then-price in `_depositWithMin/mint` remains the recommended hardening if an open-deposit vault is ever run at sustained full utilization.

Issue L-03: `mintPriceMaxAge` was silently a no-op inside the oracle's `clFreshWindow` RESOLVED

Severity: **Low**

Vulnerability Detail

`ChainlinkOracleVerifier.getPrice` reports `block.timestamp` as the price timestamp whenever the feed answer is younger than `clFreshWindow` (4h in the documented config), so `_freshPrice`'s age bound could never fire in that window: a 5-minute `mintPriceMaxAge` actually admitted prices up to `clFreshWindow` old. Exposure was bounded by the feed's 0.5% deviation trigger while live and the 150%/130% closed-market buffer – but the admin-facing knob did not mean what it appeared to mean.

Impact

A misleading freshness parameter, protocol-wide (the PSM and BorrowManager consume the same reads); no direct exploit.

Resolution

Semantics confirmed as intended: an answer inside `clFreshWindow` is deviation-bounded while the feed is live, so reporting it as current is sound, and bounding on the raw timestamp would push mints onto the in-house leg during normal hours. Documented on `RiskParams.mintPriceMaxAge` and `setMintPriceMaxAge`: the effective bound on the Chainlink leg is `max(mintPriceMaxAge, clFreshWindow)`; the knob bites on the in-house leg and on older Chainlink answers. No code change.

Issue L-09: `MINTER_ROLE` exclusivity was not structurally enforced on EUSD RESOLVED

Severity: **Low**

Vulnerability Detail

The `totalSupply == totalDebt` invariant assumes `EUSDManager` is the sole holder of `MINTER_ROLE` on the token, but plain `AccessControl` (no enumeration) cannot structurally enforce exclusivity, and nothing asserted it at deployment.

Impact

A second minter – granted by mistake or by a compromised admin – would silently break the supply-conservation invariant every other guarantee builds on.

Resolution

`DeployEusdRobinhood.s.sol` now asserts, on the freshly deployed token, that the manager holds `MINTER_ROLE`, that neither the deployer nor the token admin does, and that the deployer no longer holds `DEFAULT_ADMIN_ROLE`. Monitoring on `RoleGranted(MINTER_ROLE)` remains an ops item.

Issue L-10: eToken collateral dividends accrued to the manager with no claim path RESOLVED

Severity: **Low**

Vulnerability Detail

Collateral is protocol eTokens, which are dividend-bearing (`claimableRewards / claimRewards / rewardToken`). While an eToken sat as CDP collateral, its dividends accrued to the `EUSDManager` address – the current holder – but the manager had no claim, forward, or admin sweep path, so that yield was stranded for the life of every loan. This was asymmetric against the analogous `OwnMarket` escrow path, which does implement `sweepDividends`, and live-relevant: eSPY on Robinhood Chain has a `rewardToken` set.

Impact

A permanent value leak – the dividend stream on custodied collateral was unrecoverable.

Resolution

The reward model was first confirmed claim-based (rewards-per-share accumulator, non-rebasing – so `totalCollateral` cannot desync from the real balance). A new permissionless `sweepCollateralRewards(collateral)` claims the manager's accrued rewards and forwards them to the protocol treasury – the same destination rule as `OwnMarket.sweepDividends` and the borrow manager's collateral-dividend sweep – reverting `NoRewardsToSweep` when nothing is claimable. Tests cover the treasury forward against a real `EToken` dividend and the empty-sweep revert.

Issue L-12: Disabled collateral blocked defensive top-ups while liquidation stayed live RESOLVED

Severity: **Low**

Vulnerability Detail

`deposit` applied the `cfg.enabled` gate unconditionally, but `liquidate` has no enabled gate. After a legitimate de-listing `setCollateralEnabled(c, false)`, an indebted borrower could not top up collateral – their only risk-decreasing lever other than sourcing eUSD to repay – while keepers could still liquidate at the bonus.

Impact

In a drawdown during a de-listing window, a borrower holding spare eTokens was forced into an avoidable liquidation and paid the liquidation bonus to the keeper.

Resolution

`deposit` now reverts `CollateralDisabled` only when the caller has no debt on that collateral, so an existing debtor may always top up; `mint` keeps its unconditional gate, so a disabled collateral admits no new debt from anyone. The regression test walks a liquidatable debtor topping up to safety, the same debtor blocked from minting, and a debt-free holder blocked from depositing.

Issue L-13: Rewards streamed into an empty sEUSD vault were silently donated RESOLVED

Severity: **Low**

Vulnerability Detail

The OpenZeppelin v5 virtual-shares defense does not cover the protocol's own reward stream: if `transferInRewards` ran while `totalSupply == 0` (a stream before the seed deposit, or after a full exit including the seed), vested rewards made `totalAssets > totalSupply`, and ERC-4626 `deposit` then minted zero shares without reverting – the deposit was silently donated to nobody. The dead-shares seed was a deploy note, not code.

Impact

Depositors into an empty vault holding residual assets lost their deposit silently (e.g. a 1,000e18 deposit against a 1,000e18 residual minted 0 shares).

Resolution

`transferInRewards` reverts `NoSharesOutstanding` when `totalSupply() == 0`, so rewards can never land in an empty vault and the virtual-share defense is never asked to cover the protocol's own stream. Tested both before any deposit and again after a full exit; the seed deposit plus a `totalSupply() > 0` assertion remain on the deploy checklist.

Issue L-14: A bridge burn below the sEUSD unvested slice: vault-wide DoS, then a share-capture leg RESOLVED

Severity: **Low**

Vulnerability Detail

`EUSD.crosschainBurn` is allowance-free from any account under the trusted-bridge model, and `StakedEUSD` concentrates all stakers' eUSD at one address. A compromised bridge aiming its per-window `burnMaxLimit` at the vault socializes the loss across every holder, and a burn below the unvested reward slice originally made `totalAssets()` underflow-revert – bricking every entry and exit until vesting decayed. The first remedy (clamp `totalAssets` to zero) then opened ERC-4626 saturation pricing: with zero assets and live supply, a one-wei deposit minted more than half the share supply while every existing holder's `previewRedeem` returned zero, so as the slice vested back the newcomer redeemed it – the intended visible pro-rata loss had become winner-take-all for the first depositor (PoC: 20,000 eUSD redeemed for 1 wei).

Impact

Behind a compromised-bridge precondition (no bridge has limits today): first a vault-wide DoS, then – under the regressed remedy – capture of the surviving balance by whoever deposited first.

Resolution

`StakedEUSD` now overrides `maxDeposit/maxMint` to zero while `getUnvestedAmount()` `> totalAssets()`, so entries revert (`ERC4626ExceededMaxDeposit`) during loss recovery while withdrawals stay open – the vault degrades to a visible loss, keeps working, and the gate re-opens automatically as value vests back. The gate is inert in normal operation (a reward batch is a small fraction of TVL; ops rule: never stream more than `totalAssets()` in one batch), and the loss leg stays bridge-trust territory: size every per-bridge `burnMaxLimit` well below vested TVL before authorizing any transport. The regression asserts entries revert in the deficit state, redemption still executes, and the incumbent keeps its pro-rata post-vest claim – replacing the earlier test that had passed on the capture outcome.

Issue L-15: Force-execution never checked `order.expiry`

RESOLVED

Severity: **Low**

Vulnerability Detail

Both fill paths revert on expired orders and `expireOrder` exists to retire them – but neither the `forceExecuteOrder` wrapper (which checks only `status == Open`) nor `ForceExecuteLib._validateForce` read `order.expiry`. Report 3's M-11 explicitly noted the missing check and is marked Fixed, but git history shows only the price-freshness half of that fix landed – an expiry check never existed on the force path in any commit. An expired-but-unretired redeem order therefore remained a standing, pre-aged force-execution right against the approved vault pool indefinitely, and permissionless `expireOrder` could inversely front-run an owner's force execution to restart their claim window.

Impact

A lifecycle/state-machine hole rather than a drain – payout settles at `limitPrice` and requires a fresh price at or above it, so economic damage was capped.

Resolution

`_validateForce` reverts `OrderExpiredError` when `block.timestamp > order.expiry`, matching both fill paths. The pre-existing test only covered an order already retired by `expireOrder`; the new regression covers the un-retired expired order that was the actual hole. This closes Report 3's M-11's expiry half for good and ships with the next OwnMarket / ForceExecuteLib upgrade.

Issue L-16: `withdrawCollateral` with debt outstanding escaped the emergency mint-pause lever RESOLVED

Severity: **Low**

Vulnerability Detail

For an indebted position, withdrawing collateral is risk-increasing with exactly a mint's shape – the same `_freshPrice` and MCR gate – yet it checked neither `mintPaused` nor `cfg.enabled`. During a bad-oracle-price incident (a leaked signer or erroneous print, the threat the protocol's settle bands exist for), ops could throw both levers and debtors could still extract collateral against the inflated price down to MCR, leaving undercollateralized debt behind, while the identical extraction via `mint` was blocked. Worked case: a price pushed 2× lets a 100-token / \$50-debt position withdraw 62.5 tokens, leaving \$37.50 of real backing on \$50 of debt.

Impact

The incident lever did not actually stop value extraction.

Resolution

`withdrawCollateral` with `p.debt > 0` now reverts `MintingPaused` when the mint pause is set – the same operator lever, same scope as `mint`. It is deliberately not gated on `enabled`: per the L-12 resolution, that switch means "no new exposure" and must keep letting debtors reduce risk. Pure exits (repay, close, debt-free withdrawal) stay ungated; together with the halt and trading-pause gates in `_freshPrice`, every risk-increasing action on the manager now answers to the same switches.

Issue L-17: A code-less incentives controller bricked every sEUSD transfer despite the try/catch RESOLVED

Severity: **Low**

Vulnerability Detail

`StakedEUSD._update` wraps the controller's `handleAction` in `try/catch` precisely so a controller fault can never block sEUSD transfers – but for a high-level call to an address with no code, solc's `extcodesize` check reverts in the caller's frame, outside what `try/catch` can catch. `setIncentivesController` accepted any address, so an EOA, a typo, or a not-yet-deployed CREATE address bricked every transfer, mint, and burn – deposits, withdrawals, and money-market liquidations of sEUSD – until an admin reset it.

Impact

A vault-wide transfer DoS from an admin misconfiguration, directly falsifying the wrapper's stated guarantee. Admin-recoverable.

Resolution

`setIncentivesController` reverts `ControllerNotContract` for any non-zero target with no code, landed together with the M-02 wiring guards. The unit test sets an EOA as controller and asserts the revert.

Issue L-18: `haltAsset` price is operator-set, unbounded, and permanent

ACKNOWLEDGED

Severity: Low

Vulnerability Detail

`haltAsset(asset, price)` is an instant-role, irreversible action with no bound and no feed cross-check – and since the M-04 fix, the halt price is the sole liquidation/redemption price for every eUSD position in that collateral, a path the existing `haltRedeemAddress` containment does not touch. `haltAsset(SPY, 1)` – an operator-key compromise or a decimals fat-finger – would let any account liquidate victims' full collateral for dust, or walk the redemption list taking each position's collateral for a few hundred wei, with no un-halt and no delayed-ADMIN lever. A second wing: for a halted asset the reserve path marks exposure at the halt price but the reserve at the live wrapper price, so a low halt price manufactures skimmable phantom "surplus".

Impact

An operator-key compromise or misconfiguration becomes a collateral-seizure vector across every CDP in the halted asset, irreversibly.

Resolution

Acknowledged – `VaultManager` is immutable, so containment is operational: operator-key hygiene, monitoring on `haltAsset` parameters, and funding `haltRedeemAddress` approvals only after halt-price review. Because `EUSDManager` is upgradeable, a manager-side containment is recorded as available hardening: store the last live price per collateral and revert `HaltPriceOutOfBand` when a halt price falls outside a wide band of it, with a delayed-ADMIN override for legitimately out-of-band halts.

Issue L-21: Under in-house signer compromise the PSM ratio's damage bound is $\approx 2 \times \text{bandBps}$ and the jump guard is walkable

ACKNOWLEDGED

Severity: Low

Vulnerability Detail

The PSM ratio is the quotient of two independently band-checked in-house legs for one physical stock: the anchor band and price cache are per ticker, and every underlying/wrapper pair registers against the same aggregator. Off-hours (feed silent beyond `clSilence`), a compromised signer can push the two legs to opposite band edges – both individually valid – deriving a ratio up to $\approx 2 \times$ the band off the truth (17.4% at 800 bps). The per-operation `ratioJumpBoundBps` guard is anchored to `lastUsedRatio`, which the same actor advances: one multicall of back-dated attestations interleaved with dust PSM operations walks the baseline to the target in a single block, and a full round-trip extracts $\approx (1+b)/(1-b) - 1$ of deposited wrapper per cycle from the reserve.

Impact

The documented compromised-signer damage cap (Report 3's I-01: "the band") is off by roughly $2 \times$ for derived PSM ratios, and the ratio-jump guard – the load-bearing mitigation for `psmRedeem`'s accepted stale-mark tolerance – does not bind against this actor. The actor is the in-house KMS signing key, an accepted and instantly revocable trust root.

Resolution

Acknowledged, no live code change: the clean fix – anchoring the jump guard to the Chainlink-implied ratio – needs a getter on the verifier, and both `ChainlinkOracleVerifier` and `AssetRegistry` (which holds the guard state) are deployed and non-upgradeable, while `OwnMarket` only ever sees the served prices. The fix is folded into the next oracle/PSM deployment (anchor the guard to the Chainlink ratio, or key the in-house cache by aggregator so shared-feed tickers cannot carry divergent pushes), and the Report 3's I-01 damage-cap wording is corrected

wherever quoted. Containment is unchanged and already in place: instant
`removeSigner` plus off-hours feed/ratio monitoring.

Issue I-05: `EUSD.crosschainBurn` burns from an arbitrary `from` under the trusted-bridge model ACKNOWLEDGED

Severity: [Info](#)

Vulnerability Detail

The ERC-7802 `crosschainBurn(from, amount)` entry point is allowance-free and consent-free: authorization is solely the caller's per-bridge burn rate limit, so any admin-authorized bridge can destroy any holder's eUSD up to its per-window `burnMaxLimit`. The NatSpec claim that a bridge "only burns from the user who asked it to bridge" is an off-chain trust assumption, not an on-chain guarantee.

Impact

A compromised bridge's per-window limit is a theft budget against arbitrary holders – the standard xERC20 / ERC-7802 trusted-bridge trade-off.

Resolution

Accepted as-is: bridge authorization is admin-gated (`setBridgeLimits`), rate-limited, and launch-disabled (no bridge limits set, `maxNetBridgedIn = 0`). Recorded so the trust assumption is explicit, the NatSpec wording is corrected, and the model is re-reviewed whenever a transport is actually authorized. The plain `EUSD.burn` is `msg.sender`-only and carries no confused-deputy path.

Issue I-06: Bridge burn+mint pairing evades the `netBridgedIn` global cap

ACKNOWLEDGED

Severity: [Info](#)

Vulnerability Detail

A compromised bridge can burn X from a victim (allowance-free, decrementing `netBridgedIn`) and mint X to itself in the same window: net stays flat, the global cap never fires, and total supply is unchanged – invisible to supply monitoring. Widened on re-review: `netBridgedIn` is net across all bridges with no floor, so organic outflows through any honest bridge permanently grant every other bridge cumulative mint headroom – a compromised bridge's total unbacked mint is bounded by total historical outflow plus the cap, not by the cap alone.

Impact

The global ceiling bounds net inflation, not cumulative theft; per-window `min(mintMaxLimit, burnMaxLimit)` is the real theft budget.

Resolution

Accepted within the trusted-bridge model (I-05) – recorded so limit sizing and monitoring treat paired burn+mint flows as a theft budget rather than relying on the net accumulator. Fix direction if bridging is ever armed: track net per bridge (capping each bridge's cumulative net) instead of one global signed accumulator.

Issue I-09: Zero-amount `crosschainMint/crosschainBurn` let any EOA emit spoofed bridge events RESOLVED

Severity: [Info](#)

Vulnerability Detail

`_consumeLimit(_, 0)` passed even with a zeroed bridge config (`0 > 0` is false), so any EOA could call the ERC-7802 entry points with a zero amount and emit genuine `CrosschainMint/CrosschainBurn` events naming itself as the "bridge" – polluting exactly the event surface bridge monitoring and indexers watch.

Impact

No fund impact; noise injected into the bridge-monitoring event surface.

Resolution

Both ERC-7802 entry points now revert `ZeroAmount`. Test:

```
test_crosschain_zeroAmount_reverts.
```

Issue I-10: `setBridgeLimits` reset remaining capacity to the new maxima – an instant window refill RESOLVED

Severity: [Info](#)

Vulnerability Detail

Any limit update – including a *lowering* – reset the bridge's remaining capacity to the new maxima, handing it an instant full window on top of what it had just spent: a momentary 2× per-window throughput, at its worst exactly during an incident when limits are being tightened.

Impact

A bridge's per-window budget could momentarily double at precisely the moment ops tried to reduce it.

Resolution

A fresh authorization starts with a full window; an update to a live bridge settles accrued capacity via `_available` and clamps to the new maxima, never refilling. Three tests cover lowering mid-window (clamps, no refill), raising mid-window (no refill), and re-authorization after zero (full window).